

**PRODUCING ELECTRONIC
MUSIC BOX USING SYSTICK
TIMER AND PIT MODULE**

NAME: KELLY ATIENO DIANGÁ

Contents

AIM.....	3
PART ONE: TONE GENERATOR	3
INTRODUCTON.....	3
PIT FOR TONE GENERATION	3
SYSTICK TIMER FOR DURATION	4
.....	4
PLAYTONE FUNCTION	4
FULL SOURCE CODE TO TEST THE PLAYTONE FUNCTION	5
PART TWO:MELODY PLAYER PROGRAM.....	7
HAPPY BIRTHDAY MELODY TABLE	8
ADJUSTED MAIN FUNCTION CODE	9
PART THREE:LYRICS PRINTING	10
ADJUSTED MAIN FUNCTION CODE	10
FULL SOURCE CODE	11
WEBLINK TO VIDEO DEMONSTRATION	13

AIM

To produce an electronic music box by making use of SysTick timer and Programmable Interrupt Timer Module

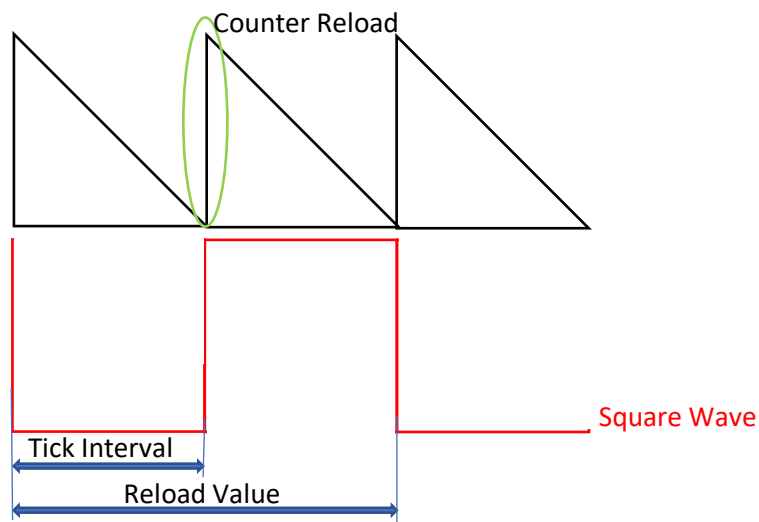
PART ONE: TONE GENERATOR

INTRODUCTION

Development of tone generator routine using SysTick and PIT module.

PIT FOR TONE GENERATION

To use the PIT, the frequency entered by the user needs to be converted to ticks. This can be done as shown below;



$$\text{Frequency} = \frac{\text{Bus Frequency}}{\text{PIT reload value}}$$

$$\text{But Tick interval} = \frac{\text{PIT reload value}}{2}$$

$$\text{Therefore Tick Interval} = \frac{\text{Bus Frequency}}{2 \times \text{PIT reload value}}$$

The tick interval will then be used to control the tone produced by the playTone using the function;

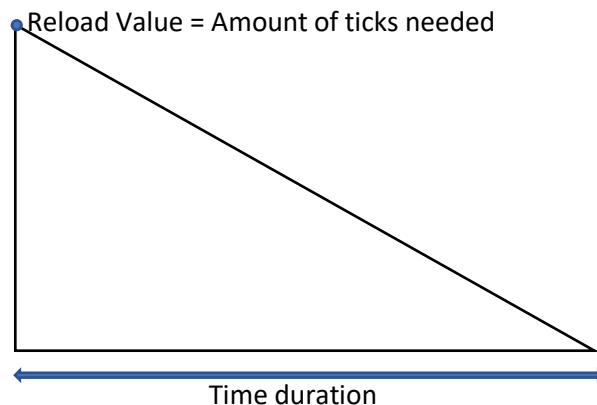
The higher the frequency, the lower the tick interval resulting in a higher pitch of sound produced and the lower the frequency, the higher the tick value hence lower pitch sound.

SYSTICK TIMER FOR DURATION

BACKGROUND STUDY

Systick Timer gives us a countdown timer. That is, for every clock cycle, it will reduce the value of counter by 1. The counter automatically counts down on its own in the background once the value reaches 0, it triggers an interrupt.

SYSTICK OPERATION



$$\text{Ticks Needed} = \frac{\text{Time duration}}{\text{Tick duration}}$$

$$\text{Tick duration} = \left(\frac{1}{48\,000\,000}\right) \text{ seconds.}$$

Using the SysTick Timer to produce the delay in the playTone Function

To tackle this problem, instead of setting the reload value for the SysTick timer then having it reduce by one for every clock cycle, I used two functions that actually increase the set tick value by one for every clock cycle and then compares it to the set delay as explained below.

I set the SysTick Handler to increase the tick value by 1 every time the SysTick happens. I then created a delay function that takes in the time duration in milliseconds. In this function, the tick value is reset to 0 and starts counting until value is equal to the duration entered, after which the delay stops.

The delay function is then called in the playTone function to set the duration in which the tone is going to play.

I also disabled the Nested Vector Interrupt Controller for the specified Irq to prevent the tone from playing infinitely.

PLAYTONE FUNCTION

Produces a specific tone for a specified duration of time by taking in two variables: Frequency and duration.

```
void playTone(int frequency, float duration)
{
    /*GENERATING TONE using PIT*/
    int tick_interval;
    //get the tick interval
```

```

        tick_interval = ((48000000)/(2*frequency));

        //Get the tone by configuring PIT channel 0 and calculated tick
interval
        initialisePIT(0,tick_interval);

        /*SETTING DURATION USING SYSTICK*/
        tick = 0;
        //Convert time to milliseconds
        int ms_duration;
        ms_duration = duration*1000;

        //Disable the Nested Vector Interrupt Controller for the
specified Irq
        while (tick<ms_duration);
        NVIC_DisableIrq(INT_PIT0);

        //Set the duration by which the tone is supposed to play
        delayMS(ms_duration);
    }

```

FULL SOURCE CODE TO TEST THE PLAYTONE FUNCTION

Below is the full program to test out the playtone function so as to make sure the tone changes with increase or decrease in frequency and plays for the specified duration of time.

```

#include "derivative.h" /* include peripheral declarations */
#include "utilities.h" /*this is where port enable and disable interrupts
is found*/
#include "clock.h" /*Set time at the specific frequency of 48 MHz */

#define BEEPER_MASK (1<<0)//Use Port E pin 0

unsigned int tick = 0; //Used by the SysTick Handler to increase the tick
value and delay function to generate duration

/** Initialises a PIT channel
 *
 * @param channel - channel (0-3) to configure
 * @param interval - PIT interval (re-load value+1)
 */
void initialisePIT(int channel, uint32_t interval) {
    // Enable clock to PIT
    SIM_SCGC6 |= SIM_SCGC6_PIT_MASK;
    // Enable PIT module
    PIT_MCR = PIT_MCR_FRZ_MASK;
    // Set reload value
    PIT_LDVAL(channel) = interval-1;
    // Enable this channel with interrupts
    PIT_TCTRL(channel) = PIT_TCTRL_TEN_MASK|PIT_TCTRL_TIE_MASK;
    // Enable PITn interrupts in NVIC
    NVIC_EnableIrq(INT_PIT0+channel);
}

/*
 * Set period(interrupt rate)
 * Enable Interrupts
 * Enable Timer
 */
uint32_t SysTick_Config(uint32_t ticks) {

```

```

    if ((ticks - 1) > SysTick_RVR_RELOAD_MASK)
    {
        /* Reload value impossible */
        return (1);
    }

    /* Set reload register */
    SYST_RVR = SysTick_RVR_RELOAD(ticks-1);

    /* Set Priority for SysTick Interrupt */
    NVIC_SetIrqPriority (INT_SysTick, (1<<4) - 1);

    /* Load the SysTick Counter Value */
    SYST_CVR = 0;

    /* Configure SysTick */
    SYST_CSR =
        SysTick_CSR_CLKSOURCE_MASK |    // Use system core clock
        SysTick_CSR_TICKINT_MASK   |    // Enable interrupts
        SysTick_CSR_ENABLE_MASK;      // Enable timer

    /* Function successful */
    return (0);
}

/*
 * PIT Channel 0 Interrupt Handler
 * Clear Interrupt Flag
 */
void PIT_Ch0_IRQHandler(void)
{
    // Toggle the pin
    GPIOE_PTOR = BEEPER_MASK;
    // Clear the interrupt flag
    PIT_TFLG0 = PIT_TFLG_TIF_MASK;
}

/*
 * SysTick Handler
 * System Timer interrupt handler to increase the tick value
 */
void SysTick_Handler(void)
{
    tick++; //Every time SysTick happens increase tick value by 1
}

/*
 * Generates delay for specific duration by comparing tick value with set
delay
 */
void delayMS(unsigned int delay)
{
    tick=0;
    while(tick<delay);
}

/*
 * Produce time for Specific duration of time
 */

```

```

void playTone(int frequency, float duration)
{
    /*GENERATING TONE using PIT*/
    int tick_interval;
    //get the tick interval
    tick_interval = ((48000000)/(2*frequency));

    //Get the tone by configuring PIT channel 0 and calculated tick
interval
    initialisePIT(0,tick_interval);

    /*SETTING DURATION USING SYSTICK*/
    tick = 0;
    //Convert time to milliseconds
    int ms_duration;
    ms_duration = duration*1000;

    //Disable the Nested Vector Interrupt Controller for the
specified Irq
    while (tick<ms_duration);
    NVIC_DisableIrq(INT_PIT0);

    //Set the duration by which the tone is supposed to play
    delayMS(ms_duration);
}

int main(void)
{
    clock_initialise();
    //Initialise Port E clock
    SIM_SCGC5 |= SIM_SCGC5_PORTE_MASK;
    //Set Port E pin 0 as a simple port pin
    PORTE_PCR0 = PORT_PCR_MUX(1);

    //Set Beeper as Output
    GPIOE_PDDR = BEEPER_MASK;

    //Initialise the SysTick
    SysTick_Config(48000);

    for(;;) {

        //Test playTone Function
        playTone(300,2);
        __asm("wfi");
    }

    return 0;
}

```

PART TWO:MELODY PLAYER PROGRAM

The following part deals with generating a melody of a song by repeating the playTone function with different frequency and duration. I chose to go with the Happy Birthday song for this part. Each chord has a specific frequency that can generate the intended tone. The beats of the song are then converted into seconds that are then used in the playTone function. The following table shows the frequencies and duration used to play the happy birthday tune.

HAPPY BIRTHDAY MELODY TABLE

TONE	FREQUENCY	LENGTH(BEATS)	DURATION	LYRICS
D	293	$\frac{3}{4}$	0.4167	Hap
D	293	$\frac{1}{4}$	0.1389	Py
E	329	1	0.5556	Birth
D	293	1	0.5556	Day
G	392	1	0.5556	To
F	349	2	1.1112	You!
D	293	$\frac{3}{4}$	0.4167	Hap
D	293	$\frac{1}{4}$	0.1389	Py
E	329	1	0.5556	Birth
D	293	1	0.5556	Day
A	220	1	0.5556	To
G	392	2	1.1112	You!
D	293	$\frac{3}{4}$	0.4167	Hap
D	293	$\frac{1}{4}$	0.1389	Py
D	293	1	0.5556	Birth
B	246	1	0.5556	Day
G	392	1	0.5556	Dear
F	349	1	0.5556	Ba
E	329	1	0.5556	by
C	261	$\frac{3}{4}$	0.4167	Hap
C	261	$\frac{1}{4}$	0.1389	Py
B	246	1	0.5556	Birth

G	392	1	0.5556	Day
A	220	1	0.5556	To
G	392	2	1.1112	You!

In order to play the birthday tune, I effectively stored the melody information (frequency and duration) in the memory using arrays.

ADJUSTED MAIN FUNCTION CODE

The following program shows how I adjusted the main function so as to play the entire melody. It includes the arrays to store the frequencies and durations as well as a for loop to play the entire song.

```
int main(void)
{
    clock_initialise();
    SIM_SCGC5 |= SIM_SCGC5_PORTE_MASK;
    PORTE_PCR0 = PORT_PCR_MUX(1);

    //Set Beeper as Output
    GPIOE_PDDR = BEEPER_MASK;

    //Initialise the SysTick
    SysTick_Config(48000);

    /*HAPPY BIRTHDAY FREQUENCY AND DURATION*/
    // Array for frequency
    int SONG_FREQ[] = {293,293,329,293,392,349,293,293,329,293,220,392,
293,293,293,246,392,349,329,261,261,246,392,220,392}; //Happy birthday
    // Array for duration
    float SONG_DURATION[] = {0.4167,0.1389,0.5556,0.5556,0.5556,1.1112,
0.4167,0.1389,0.5556,0.5556,0.5556,1.1112,0.4167,0.1389,0.5556,0.5556,0.555
6,0.5556,0.5556,0.4167,0.1389,0.5556,0.5556,0.5556,1.1112}; //Happy Birthday

    for(;;)
    {
        //Use a for loop to play the whole song
        int i = 0;
        for (i = 0; i < 25; i++)
        {
            NVIC_EnableIrq(INT_PIT0); //Enable interrupt
            while ( i < SONG_FREQ)
            {
                // Use playTone function to play the birthday song
                playTone(SONG_FREQ[i], SONG_DURATION[i]);
                i++;
                NVIC_DisableIrq(INT_PIT0); // disable interrupts
            }
            __asm("wfi"); //Wait for interrupt to occur
        }
    }

    return 0;
}
```

PART THREE:LYRICS PRINTING

This part of the lab deals with simultaneously printing out the lyrics via the serial terminal as the melody plays. I stored the lyrics in a character array and added “fflush(stdout);” after the printf to force the string to print immediately without waiting for “\n”. I also included string.h,stdio.h and stdlib.h header files to enable me to print the lyrics.

ADJUSTED MAIN FUNCTION CODE

The following lines of codes shows how I adjusted the main function from playing the tune only to simultaeously palyng the tune and printing out the lyrics in the termite.

```
int main(void)
{
    clock_initialise();
    //Initialise Port E clock
    SIM_SCGC5 |= SIM_SCGC5_PORTE_MASK;
    //Set Port E pin 0 as a simple port pin
    PORTE_PCR0 = PORT_PCR_MUX(1);

    //Set port E as Output for the beeper
    GPIOE_PDDR = BEEPER_MASK;

    //Initialise the SysTick
    SysTick_Config(48000);

    // Array for frequency
    int SONG_FREQ[] = {293,293,329,293,392,349,293,293,329,293,220,392,
293,293,293,246,392,349,329,261,261,246,392,220,392};

    // Array for duration
    float SONG_DURATION[] = {0.4167,0.1389,0.5556,0.5556,0.5556,1.1112,
0.4167,0.1389,0.5556,0.5556,0.5556,1.1112,0.4167,0.1389,0.5556,0.5556,0.5556,0.5556,
0.5556,0.4167,0.1389,0.5556,0.5556,0.5556,1.1112};

    //LYRICS:Happy Birthday to you!happy birthday to you!Happy birthday dear
    baby,happy birthday to you!
    char LYRICS[26][13] = { "Hap", "py ", " Birth", "day", " to", " you! \n \n",
"Hap", "py ", " Birth", "day", " to", " you! \n ", " \n Hap", "py ", " Birth",
"day", " dear", " ba", "by \n", "\n Hap", "py ", " Birth", "day", " to", " you!"};

    for(;;)
    {
        //SONG TITLE
        printf( "\n\t HAPPY BIRTHDAY MELODY\n\t_____ \n\n");
        int i = 0;
        for (i = 0; i < 25; i++)
        {
            NVIC_EnableIrq(INT_PIT0); //enable interrupts for channel 0
            while ( i <= 25)
            {
                playTone(SONG_FREQ[i], SONG_DURATION[i]); // calling out the
playTone function to play the Array
                printf("%s", LYRICS[i]); //print the lyrics
                fflush(stdout); //forces the string to print immediately
                i++;
                NVIC_DisableIrq(INT_PIT0); // disable interrupts for channel 0
            }

        }
        __asm("wfi");

        return 0;
    }
}
```

FULL SOURCE CODE

This is the full code used in this lab to create the electronic music box.

```
#include "derivative.h" /* include peripheral declarations */
#include "utilities.h" /*this is where port enable and disable interrupts is found*/
#include "clock.h"
#include "string.h"
#include <stdio.h> // inputs and outputs
#include <stdlib.h> // standard library to use fflush(stdout)
#define BEEPER_MASK (1<<0) //Use Port E pin 0

unsigned int tick = 0; //Used by the SysTick Handler to increase the tick value and
delay function to generate duration

/** Initialises a PIT channel
 *
 * @param channel - channel (0-3) to configure
 * @param interval - PIT interval (re-load value+1)
 */
void initialisePIT(int channel, uint32_t interval) {
    // Enable clock to PIT
    SIM_SCGC6 |= SIM_SCGC6_PIT_MASK;
    // Enable PIT module
    PIT_MCR = PIT_MCR_FRZ_MASK;
    // Set reload value
    PIT_LDVAL(channel) = interval-1;
    // Enable this channel with interrupts
    PIT_TCTRL(channel) = PIT_TCTRL_TEN_MASK|PIT_TCTRL_TIE_MASK;
    // Enable PITn interrupts in NVIC
    NVIC_EnableIrq(INT_PIT0+channel);
}

/*
 * Set period(interrupt rate)
 * Enable Interrupts
 * Enable Timer
 */
uint32_t SysTick_Config(uint32_t ticks) {

    if ((ticks - 1) > SysTick_RVR_RELOAD_MASK)
    {
        /* Reload value impossible */
        return (1);
    }

    /* Set reload register */
    SYST_RVR = SysTick_RVR_RELOAD(ticks-1);

    /* Set Priority for SysTick Interrupt */
    NVIC_SetIrqPriority (INT_SysTick, (1<<4) - 1);

    /* Load the SysTick Counter Value */
    SYST_CVR = 0;

    /* Configure SysTick */
    SYST_CSR =
        SysTick_CSR_CLKSOURCE_MASK |    // Use system core clock
        SysTick_CSR_TICKINT_MASK |      // Enable interrupts
        SysTick_CSR_ENABLE_MASK;        // Enable timer
}
```

```

    /* Function successful */
    return (0);
}

/*
 * PIT Channel 0 Interrupt Handler
 * Clear Interrupt Flag
 */
void PIT_Ch0_IRQHandler(void)
{
    // Toggle the pin
    GPIOE_PTOR = BEEPER_MASK;
    // Clear the interrupt flag
    PIT_TFLG0 = PIT_TFLG_TIF_MASK;
}

/*
 * SysTick Handler
 * System Timer interrupt handler to increase the tick value
 */
void SysTick_Handler(void)
{
    tick++; // Every time SysTick happens increase tick value by 1
}

/*
 * Generates delay for specific duration by comparing tick value with set delay
 */
void delayMS(unsigned int delay)
{
    tick = 0;
    while (tick < delay);
}

/*
 * Produce time for Specific duration of time
 */
void playTone(int frequency, float duration)
{
    /* GENERATING TONE using PIT */
    int tick_interval;
    // get the tick interval
    tick_interval = ((48000000) / (2 * frequency));

    // Get the tone by configuring PIT channel 0 and calculated tick interval
    initialisePIT(0, tick_interval);

    /* SETTING DURATION USING SYSTICK */
    tick = 0;
    // Convert time to milliseconds
    int ms_duration;
    ms_duration = duration * 1000;

    // Disable the Nested Vector Interrupt Controller for the specified Irq
    while (tick < ms_duration);
    NVIC_DisableIrq(INT_PIT0);

    // Set the duration by which the tone is supposed to play
    delayMS(ms_duration);
}

int main(void)
{
    clock_initialise();
    // Initialise Port E clock
    SIM_SCGC5 |= SIM_SCGC5_PORTE_MASK;
    // Set Port E pin 0 as a simple port pin

```

```

PORTE_PCR0 = PORT_PCR_MUX(1);

//Set port E as Output for the beeper
GPIOE_PDDR = BEEPER_MASK;

//Initialise the SysTick
SysTick_Config(48000);

// Array for frequency
int SONG_FREQ[] = {293,293,329,293,392,349,293,293,329,293,220,392,
293,293,293,246,392,349,329,261,261,246,392,220,392};

// Array for duration
float SONG_DURATION[] = {0.4167,0.1389,0.5556,0.5556,0.5556,1.1112,
0.4167,0.1389,0.5556,0.5556,0.5556,1.1112,0.4167,0.1389,0.5556,0.5556,0.5556,0.5556,
0.5556,0.4167,0.1389,0.5556,0.5556,0.5556,1.1112};

//LYRICS:Happy Birthday to you!happy birthday to you!Happy birthday dear
baby,happy birthday to you!
char LYRICS[26][13] = { "Hap", "py ", " Birth", "day", " to", " you! \n \n",
"Hap", "py ", " Birth", "day", " to", " you! \n ", " \n Hap", "py ", " Birth",
"day", " dear", " ba", "by \n", "\n Hap", "py ", " Birth", "day", " to", " you!"};

for(;;)
{
    //SONG TITLE
    printf( "\n\t HAPPY BIRTHDAY MELODY\n\t_____ \n\n");
    int i = 0;
    for (i = 0; i < 25; i++)
    {
        NVIC_EnableIrq(INT_PIT0); //enable interrupts for channel 0
        while ( i <= 25)
        {
            playTone(SONG_FREQ[i], SONG_DURATION[i]); // calling out the
playTone function to play the Array
            printf("%s", LYRICS[i]); //print the lyrics
            fflush(stdout); //forces the string to print immediately
            i++;
            NVIC_DisableIrq(INT_PIT0); // disable interrupts for channel 0
        }

    }

    __asm("wfi");

    return 0;
}
}

```

WEBLINK TO VIDEO DEMONSTRATION

Below is the link to the demonstration video for the electronic music box.

<https://youtu.be/sPVJG6AJdBc>